

OSSP PRACTICAL Q6:-

1. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Code:-

Server code:-

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
#include <sys/stat.h>
```

```
#include <errno.h>
```

```
#define SERVER_FIFO "/tmp/server_fifo"
```

```
#define BUFFER_SIZE 256
```

```
int main() {
```

```
    char buffer[BUFFER_SIZE];
```

```
    // Remove old FIFO if it exists
```

```
    unlink(SERVER_FIFO);
```

```
// Create named pipe

if (mkfifo(SERVER_FIFO, 0666) == -1) {

    perror("mkfifo");

    exit(EXIT_FAILURE);

}


printf("Server started...\n");

printf("Waiting for clients...\n");


while (1) {


    // Open FIFO for reading

    int fd = open(SERVER_FIFO, O_RDONLY);

    if (fd == -1) {

        perror("open");

        continue;

    }


    // Read message from client

    ssize_t bytesRead = read(fd, buffer, BUFFER_SIZE - 1);
```

```
if (bytesRead > 0) {  
    buffer[bytesRead] = '\0';  
  
    printf("Client message: %s\n", buffer);  
  
    // Process the message  
    char response[BUFFER_SIZE];  
  
    snprintf(response, BUFFER_SIZE,  
        "Server received: %s", buffer);  
  
    printf("Response: %s\n", response);  
  
    /*  
    * In a simple FIFO example, print the response.  
    * A production client-server design would use  
    * a separate FIFO for each client's response.  
    */  
  
    printf("-----\n");  
}
```

```
        close(fd);  
    }  
  
    unlink(SERVER_FIFO);  
  
    return 0;  
}
```

Client Code:-

```
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <string.h>  
  
#include <unistd.h>  
  
#include <fcntl.h>  
  
  
#define SERVER_FIFO "/tmp/server_fifo"  
  
#define BUFFER_SIZE 256  
  
int main() {  
  
    char message[BUFFER_SIZE];
```

```
printf("Enter message: ");

fgets(message, BUFFER_SIZE, stdin);


// Remove newline
message[strcspn(message, "\n")] = '\0';


// Open server FIFO for writing
int fd = open(SERVER_FIFO, O_WRONLY);


if (fd == -1) {
    perror("open");
    exit(EXIT_FAILURE);
}


// Send message to server
write(fd, message, strlen(message) + 1);


printf("Message sent to server.\n");


close(fd);


return 0;
```

```
}
```

OUTPUT:-

```
        printf("Response: %s\n", response);

        /*
         * In a simple FIFO example, print the response.
         * A production client-server design would use
         * a separate FIFO for each client's response.
         */

        printf("-----\n");
    }

    close(fd);
}

unlink(SERVER_FIFO);

return 0;
}
```

```
apple@apples-MacBook-Pro-2 OSSP % cd fifo
apple@apples-MacBook-Pro-2 fifo % nano pracq6prod.c
apple@apples-MacBook-Pro-2 fifo % gcc pracq6prod.c -o pracq6
apple@apples-MacBook-Pro-2 fifo % nano pracq6client.c
apple@apples-MacBook-Pro-2 fifo % gcc pracq6client.c -o pracq6client
apple@apples-MacBook-Pro-2 fifo % ./pracq6
Server started...
Waiting for clients...
Client message: hi
Response: Server received: hi
-----
```

```
apple@apples-MacBook-Pro-2 ~ % cd OSSP
apple@apples-MacBook-Pro-2 OSSP % cd FIFO
apple@apples-MacBook-Pro-2 FIFO % ls
pracq6      pracq6client  pracq6client.c  pracq6prod.c
apple@apples-MacBook-Pro-2 FIFO % ./pracq6client
Enter message: hi
Message sent to server.
apple@apples-MacBook-Pro-2 FIFO %
```